There’s a lot going on in this example, but it gives some nice benefits:

- You didn’t have to give your Facebook credentials to dogphotos.com. You simply signed in to Facebook as normal.
- You had control of which details dogphotos.com could access on your behalf via the Facebook photos API.
- You didn’t have to give dogphotos.com any of your identity information (though in practice, this is often requested).

Effectively, you delegated your access of the Facebook photos API to dogphotos.com. This approach is why OAuth 2.0 is described as an authorization protocol, not an authentication protocol. dogphotos.com doesn’t know your identity on Facebook; it is authorized only to access the photos API on behalf of someone.

OAuth 2.0 authorization flows and grant types

The OAuth 2.0 example shows in this section uses a common flow or *grant type*, as it’s called in OAuth 2.0, for obtaining a token from an identity provider. Oauth 2.0 defines several grant types and extensions, each designed for a different scenario:

- *Authorization code*—This is the flow I described in figure 25.5, in which an application uses the combination of an authorization code and a secret to retrieve a token.
- *Proof Key for Code Exchange (PKCE)*—This is an extension to the authorization code that you should always favor, if possible, as it provides additional protections against certain attacks, as described in the RFC at https://www.rfc-editor.org/rfc/rfc7636.
- *Client credentials*—This is used when no user is involved, such as when you have an API talking to another API.

Many more grants are available (see https://oauth.net/2/grant-types), and each grant is suited to a different situation. The examples are the most common types, but if your scenario doesn’t